

Laboratorio 06 · Write concern, read concern, prioridades y tags

Duración: 35 minutos · Carpeta de trabajo: `entorno/`

Objetivos

Vas a comprobar en la práctica qué garantiza cada nivel de *write concern*, a dirigir lecturas a los secundarios con *read preference* y tags, a cambiar las prioridades para decidir qué nodo es primario y a configurar un secundario oculto y retrasado.

Todo el laboratorio se hace desde el toolbox: `docker compose exec toolbox bash` y `mongosh "$RS"`.

Parte A · Write concern

A.1 Valores por defecto

```
> db.adminCommand({ getDefaultRWConcern: 1 })
```

Desde MongoDB 5.0 el write concern por defecto es `{ w: "majority" }`.

A.2 Escribir con distintos niveles

```
> use tienda
> db.pruebas.insertOne({ n: 1 }, { writeConcern: { w: 1 } })
> db.pruebas.insertOne({ n: 2 }, { writeConcern: { w: "majority" } })
> db.pruebas.insertOne({ n: 3 }, { writeConcern: { w: 3, wtimeout: 3000 } })
```

A.3 ¿Qué pasa si un nodo cae?

En **otra terminal** de tu equipo, para un secundario (elige uno que no sea el primario):

```
docker compose stop mongo3
```

Vuelve a `mongosh` y repite:

```
> db.pruebas.insertOne({ n: 4 }, { writeConcern: { w: "majority" } })
> db.pruebas.insertOne({ n: 5 }, { writeConcern: { w: 3, wtimeout: 3000
} })
> db.pruebas.find({ n: 5 })
```

1. ¿Qué escrituras han funcionado? ¿Cuál ha dado error y qué error?
2. La escritura que ha fallado, ¿se ha guardado o no?

Arranca de nuevo el nodo: `docker compose start mongo3`.

▼ Ver solución

Con dos de tres nodos vivos, `w: "majority"` (2 nodos) se cumple sin problema. `w: 3` no puede cumplirse y, al vencer el `wtimeout`, devuelve un `writeConcernError` (`WriteConcernFailed / waiting for replication timed out`). Pero **el documento sí se ha escrito** en el primario y se replicará a `mongo3` cuando vuelva. El write concern no es una transacción: no deshace la escritura, solo te dice hasta dónde se ha confirmado. Por eso nunca debe usarse `wtimeout` como si fuera un rollback. Sin `wtimeout`, la escritura con `w: 3` se habría quedado esperando indefinidamente.

Parte B · Read preference y read concern

B.1 ¿Quién responde a mis lecturas?

```
> db.pedidos.find({ estado: "pagado"
}).limit(1).explain().serverInfo.host
> db.getMongo().setReadPref("secondary")
> db.pedidos.find({ estado: "pagado"
}).limit(1).explain().serverInfo.host
> db.getMongo().setReadPref("secondaryPreferred")
> db.getMongo().setReadPref("primary")
```

B.2 Read concern

```
> db.pedidos.find({ estado: "pagado" }).readConcern("local").limit(1)
> db.pedidos.find({ estado: "pagado" }).readConcern("majority").limit(1)
> db.pedidos.find({ _id: db.pedidos.findOne()._id
}).readConcern("linearizable").maxTimeMS(5000)
```

Pregunta: ¿qué problema puede tener leer de un secundario con `readConcern: "local"` justo después de escribir en el primario?

▼ Ver solución

La réplica es asíncrona: el secundario puede no haber aplicado todavía tu escritura, así que la lectura devolvería un dato anterior (no lees tus propias escrituras). Si la aplicación necesita leer sus propias escrituras desde secundarios, debe usar **sesiones causalmente consistentes** con `readConcern: "majority"` y `writeConcern: "majority"`.

`linearizable` solo se puede usar en el primario, sobre un único documento, y es la opción más cara: garantiza que lees el último valor confirmado por mayoría antes de empezar la lectura.

Parte C · Prioridades

```
> rs.conf().members.map(m => ({ host: m.host, prioridad: m.priority,
votos: m.votes })))
> db.hello().primary
```

Haz que `mongo3` sea el primario preferido:

```
> cfg = rs.conf()
> cfg.members[2].priority = 3
> rs.reconfig(cfg)
```

Espera unos segundos y comprueba quién es el primario. Después devuelve todas las prioridades a 1 y vuelve a mirar: ¿cambia el primario?

▼ Ver solución

Tras subir la prioridad, `mongo3` convoca una elección en cuanto está al día (*catch-up*) y se convierte en primario: una elección por prioridad (*priority takeover*). Al devolver las prioridades a 1, el primario **no cambia**: con prioridades iguales no hay motivo para convocar una elección, y MongoDB evita elecciones innecesarias porque cada cambio de primario provoca unos segundos sin escrituras.

Antes de seguir, comprueba quién es el primario. Si es `mongo3`, haz que deje de serlo, porque las partes D y E necesitan que sea secundario. Con `mongosh "$RS"` conectado al replica set:

```
> rs.stepDown()
> db.hello().primary
```

`rs.stepDown()` obliga al primario a dejar de serlo y a no presentarse como candidato durante 60 segundos, así que el nuevo primario será `mongo1` o `mongo2`.

Parte D · Tags y lecturas dirigidas

Vamos a etiquetar los nodos: `mongo1` y `mongo2` atienden la aplicación y `mongo3` es el nodo de informes.

```
> cfg = rs.conf()
> cfg.members[0].tags = { dc: "madrid", uso: "oltp" }
> cfg.members[1].tags = { dc: "madrid", uso: "oltp" }
> cfg.members[2].tags = { dc: "barcelona", uso: "informes" }
> rs.reconfig(cfg)
```

Lanza una consulta de informes dirigida a ese nodo:

```
> db.getMongo().setReadPref("secondary", [ { uso: "informes" } ])
> db.pedidos.explain().aggregate([ { $group: { _id: "$ciudad", total: {
$sum: "$total" } } } ]).serverInfo.host
> db.getMongo().setReadPref("primary")
```

Lo mismo se puede expresar en la cadena de conexión de la aplicación de informes:

```
mongodb://...@mongo1,mongo2,mongo3/?
replicaSet=rs0&readPreference=secondary&readPreferenceTags=uso:informes
```

Si `mongo3` fuera el primario en este momento, la consulta fallaría: no habría ningún secundario con ese tag. Piensa cómo lo resolverías.

▼ Ver solución

Se pueden encadenar varios conjuntos de tags de más a menos preferido, terminando en un conjunto vacío que significa "cualquier secundario": `setReadPref("secondary", [ { uso: "informes" }, { } ])` o, en la URI, `readPreferenceTags=uso:informes&readPreferenceTags=`. También puedes impedir que el nodo de informes llegue a ser primario dándole `priority: 0`.

Parte E · Secundario oculto y retrasado

Un secundario retrasado es una "máquina del tiempo" que protege frente a errores humanos. Configura `mongo3` con una hora de retraso... aunque en el laboratorio usaremos 60 segundos. Antes de reconfigurar, comprueba que `mongo3` **no** es el primario.

```
> cfg = rs.conf()
> cfg.members[2].priority = 0
> cfg.members[2].hidden = true
```

```
> cfg.members[2].secondaryDelaySecs = 60
> rs.reconfig(cfg)
> db.hello().hosts
```

Inserta un documento y comprueba cuándo llega a `mongo3`:

```
> db.getSiblingDB("tienda").pruebas.insertOne({ marca: "retraso", ts:
new Date() })
> exit
```

```
mongosh "mongodb://admin:CursoMongo2026@mongo3:27017/?
authSource=admin&directConnection=true" --quiet --eval
"db.getMongo().setReadPref('secondary');
db.getSiblingDB('tienda').pruebas.findOne({ marca: 'retraso' })"
```

Repite el comando cada 15 segundos hasta que aparezca el documento.

1. ¿Por qué `db.hello().hosts` ya no muestra `mongo3`?
2. ¿Qué pasa si el `oplog` cubre menos tiempo que `secondaryDelaySecs`?

Al terminar, deja `mongo3` como un secundario normal, **conservando los tags**:

```
> cfg = rs.conf()
> cfg.members[2].priority = 1
> cfg.members[2].hidden = false
> cfg.members[2].secondaryDelaySecs = 0
> rs.reconfig(cfg)
```

▼ Ver solución

Un miembro `hidden` no se anuncia a los clientes en `hello`, así que los drivers nunca le envían lecturas; solo recibe tráfico de conexiones directas. Por eso es ideal para backups o informes pesados. Un miembro oculto debe tener `priority: 0`. Si el `oplog` cubre menos tiempo que el retraso, el nodo retrasado se quedará sin las entradas que necesita y acabará en estado `RECOVERING`, exigiendo una resincronización completa. Además, conviene que un miembro retrasado tenga `votes: 0` en muchos diseños: si vota y los demás nodos caen, la mayoría para `w: "majority"` puede depender de un nodo que va una hora por detrás.